



INITDEV · TECHNICAL SHOWCASE

A forensic, evidence-backed analysis of a privacy-first relationship-wellness AI companion and its operator control plane.

Drip privately listens to each partner, builds a layered, privacy-aware memory that strictly governs what can ever be shared, and turns ongoing conversations into personalized guidance - with a human operator able to review, approve, and override the AI. This document separates what is implemented from what is proposed, with code-path evidence throughout.

Prepared by

Dev Ganugapenta

Founder, InitDev

Next.js 15 · React 19 · TypeScript

Clerk · Supabase · Pinecone · OpenAI · Stripe

98 API routes · 38 migrations · 528 commits

Evidence labels: IMPLEMENTED / PARTIAL / PROPOSED

01 Executive Assessment

WHAT IT IS & WHY IT MATTERS

Drip is an integrated AI product, not a demo or a thin API wrapper. It couples a consumer chat experience with a deep operator console, and is wired end to end to managed authentication, a relational database, a vector database, an LLM provider, billing, transactional email, and analytics.

98
API ROUTE HANDLERS

44
APP PAGES

38
SQL MIGRATIONS

528
COMMITTS (SOLE AUTHOR)

Category & maturity

Relationship-wellness AI companion (consumer SaaS + operator console). Real third-party wiring with several hardening items open: best described as an advanced pre-production SaaS.

Evidence: full stack inventory, section 05.

Audience

Partner A (initiator), Partner B (invited), and an internal Operator/Admin who reviews and steers the AI.

Evidence: `src/middleware.ts`, `partner-b-onboarding.ts`.

Why it is technically notable

A privacy-aware dual-store memory system, a human-in-the-loop approval console, and a stateful multi-session journey engine - the patterns that make AI products hard.

Evidence: sections 09-11.

Commercial status

Subscription billing implemented via Stripe. Revenue, users, and growth are **UNVERIFIED** external claims and are not asserted here.

Evidence: `src/app/api/stripe/**`.

How to read this document

Every material claim is tied to a code path. Evidence labels are used strictly: **IMPLEMENTED** present and verifiable, **PARTIAL** present but incomplete/bypassed, **PROPOSED** documented not built, **INFERRED** strongly suggested, **UNVERIFIED** external business claim, **DEPRECATED** superseded by choice. Screenshots were not captured (the app could not be kept running in this sandbox); the visual center is architecture diagrams and code-derived data. See section 06.

02 Verified Proof at a Glance

CODE-DERIVED FACTS

A small set of reproducible, code-derived facts (generated by `portfolio-showcase/scripts/inventory.mjs` into `report-data.json`). No figure here is hand-counted.

Fact	Value	Meaning
API route handlers	98	Large backend surface; 57 are operator/admin routes
App pages	44	Consumer + a 45-component admin console
SQL migrations	38	Chronological schema evolution under <code>supabase/migrations</code>
Distinct DB tables (in SQL)	~37	29 created in-repo + 8 altered-only base tables
lib modules	51	7 memory, 14 services - real business logic layer
Mapped integrations	14	Auth, DB, vector, LLM, billing, email, analytics, more
Zustand stores	5	Focused client-state separation
Scheduled crons	1	Vercel cron for session emails (+1 cron-ready)
Commits / author	528 / 1	Sole author, 2025-07 to 2026-02
Automated tests	0	Honest gap; manual diagnostic scripts only
Type-check errors	39	Non-blocking by config; concentrated in admin UI/scripts

Balanced from the start

The same inventory that shows real breadth also shows the honest gaps: zero automated tests, 39 suppressed type errors, and 910 lint warnings. These are carried openly through the document rather than hidden.

03 Product & Business Context

USER, PROBLEM, WORKFLOW, VALUE

The problem

Couples rarely have a private, judgement-free space to process their relationship. Generic chatbots forget context and, worse, could leak one partner's words to the other. Drip is built around the opposite guarantee.

The core workflow

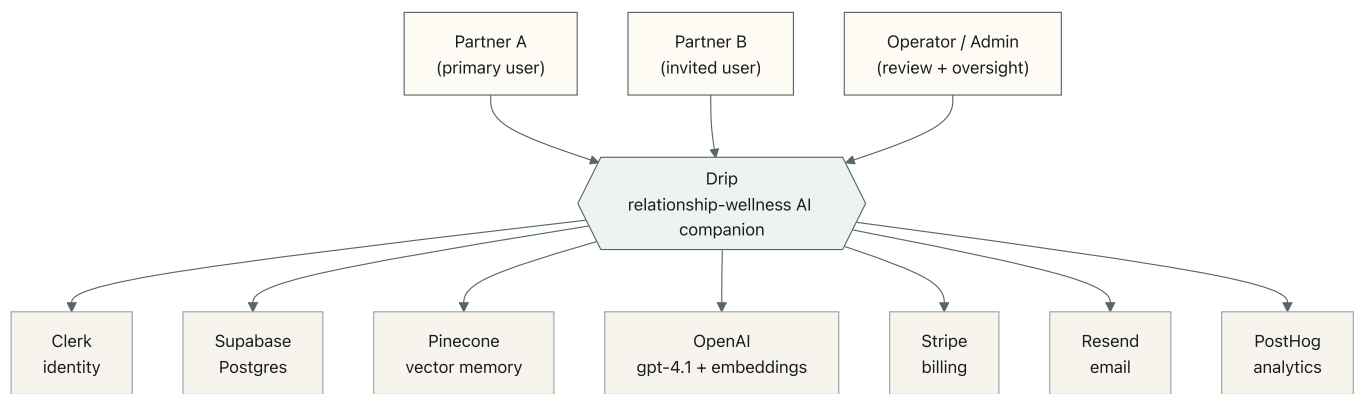
A message enters the chat API, relevant memories are retrieved by semantic search, a layered system prompt is assembled, the LLM returns a structured response, messages are persisted, and new memories are extracted, classified for privacy, and stored to both stores in the background.

Who it serves

Two partners individually and privately, plus an operator who can review conversations, draft and approve assisted replies, annotate quality, and manage AI behaviour.

Where value concentrates

Trust (privacy guarantees), retention (a memory that improves over time), and safety/defensibility (human oversight of AI output). Subscription billing monetizes the consumer side.



System context: three human roles and the seven external systems Drip integrates.

What differentiates it from a CRUD app

The domain is encoded in the data and AI layers: per-partner memory ownership, a four-value privacy store, consent-gated sharing, and a session state machine. These are modeling decisions, not boilerplate.

04 Product Surfaces

WALKTHROUGH & CAPTURE NOTE

Screenshot capture limitation (honest)

Live screenshots were not captured. The application requires a live `ClerkProvider` even for public pages, and in this sandbox the dev server was repeatedly reaped within seconds of becoming ready (the machine was concurrently running several other projects' dev servers). No screens were fabricated. A reproducible capture script is provided at `scripts/capture-screenshots.mjs` with a plan at `screenshots/screenshot-plan.md`. Surfaces below are evidenced by routes and components.

Auth & onboarding

Hosted sign-in/up, invitation acceptance, guided A/B onboarding chat. **IMPLEMENTED**

Conversational workspace

Core chat at `/chat` with structured AI replies + memory recall. **IMPLEMENTED**

Account & billing

Stripe checkout, customer portal, webhook-driven status. **IMPLEMENTED**

Operator: conversations

Draft, approve/reject, send, or take over manually. **IMPLEMENTED**

Operator: users & couples

Records, journey advancement, goals, anonymized views. **IMPLEMENTED**

Memory management

Inspect, resolve ambiguous, reset per-user memories. **IMPLEMENTED**

Prompt control

DB-backed prompts + session/email prompts with history. **IMPLEMENTED**

Journey analytics

Phase/journey dashboards (Recharts / React Flow). **IMPLEMENTED**

Reflections & feedback

Structured questionnaires + feedback pipeline. **IMPLEMENTED**

API surface distribution (98 handlers)

Admin / operator	57
Conversations & memory	10
Invitations & couple	6
Journey & reflections	5
Messaging / email / cron	5
Consent / moments	5
Billing (Stripe)	4
Webhooks	1

More than half the backend exists to give a human operator visibility and control over the AI - the signal of a serious assisted-workflow product rather than a chat toy.

Drip - Technical Showcase

04 · Product Surfaces

p. 5

05 Complete Technology Stack

HOW EACH IS ACTUALLY USED

Layer	Technology	Actual use	Maturity
Framework	Next.js 15.3.6	App Router; 44 pages, 98 route handlers (Node runtime; 1 edge route)	IMPLEMENTED
UI	React 19, Tailwind 3, Framer Motion	Consumer chat + 45-component admin panel; dedicated mobile landing	IMPLEMENTED
Client state	Zustand	5 focused stores (conversation, profile, user, onboarding, theme)	IMPLEMENTED
Auth	Clerk	Middleware route protection + admin gating (404-hide)	IMPLEMENTED
Relational DB	Supabase Postgres	38 migrations; service-role server access; RLS defined	IMPLEMENTED
Vector DB	Pinecone	Integrated llama-text-embed-v2 embeddings for memory	IMPLEMENTED
LLM	OpenAI Responses API	gpt-4.1 structured output; mini models for admin tools	IMPLEMENTED
Billing	Stripe	Checkout, portal, signature-verified webhook	IMPLEMENTED
Email	Resend	Session-transition + admin outbound messaging	IMPLEMENTED
Webhook verify	Svix / Stripe	Clerk (Svix) + Stripe constructEvent	IMPLEMENTED
Analytics	PostHog	Client instrumentation	IMPLEMENTED
Queue / cache	Upstash Redis	Declared in deps/env; no imports in src/	PARTIAL
Error tracking	Sentry	In env.example only; not wired	PROPOSED
Rate limiting	-	Not present; CORS is * on /api/*	PROPOSED

Reading maturity honestly

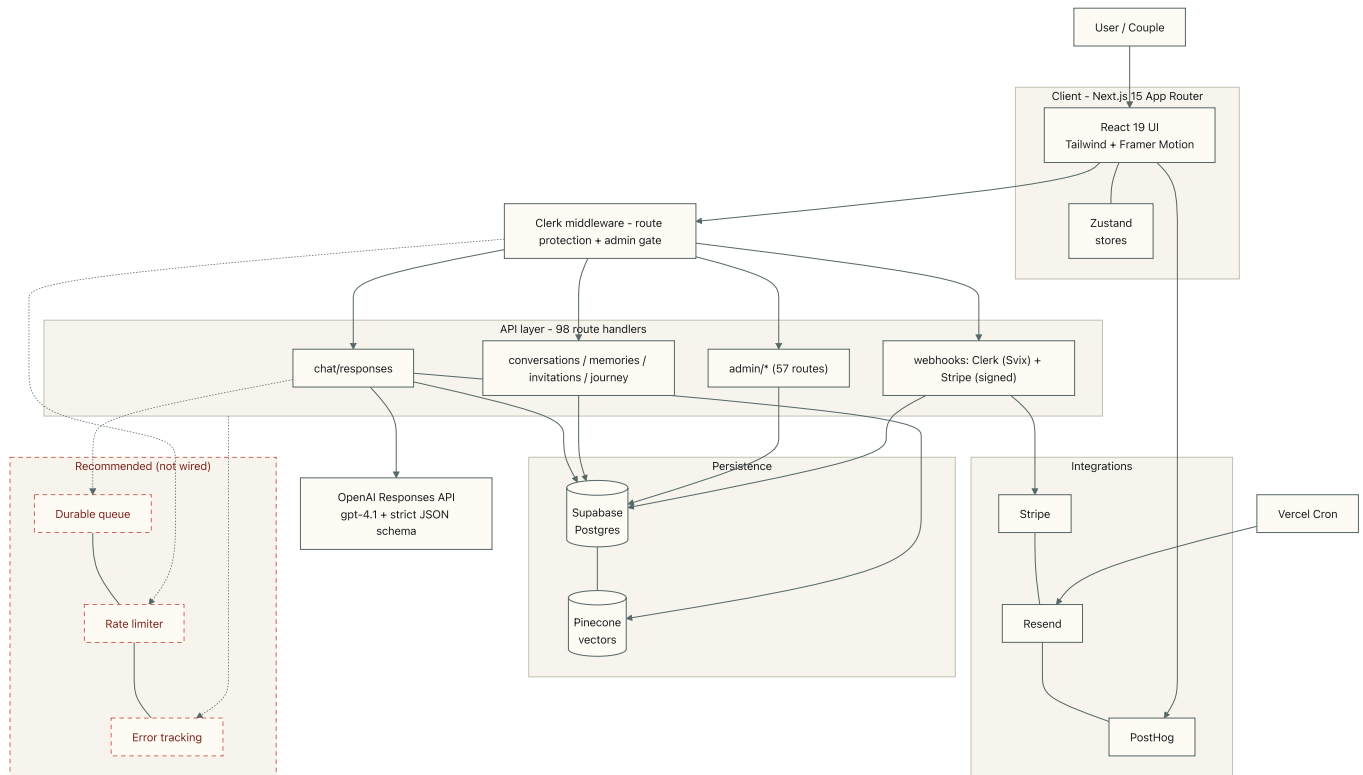
Installed packages are not proof of use. Upstash is a dependency but is never imported in src/ ; Sentry appears only in the env template. Both are labelled accordingly rather than counted as features.

06 System Architecture

REAL FLOW OF THE APPLICATION

A request passes the Clerk middleware trust boundary, is handled by a Next.js route handler using the service-role Supabase client, and - for chat - fans out to Pinecone (memory) and OpenAI (generation) before persisting messages and queuing background memory extraction. Webhooks (Clerk, Stripe) are signature-verified inbound flows. A Vercel cron drives lifecycle email.

— implemented flow - - - recommended extension

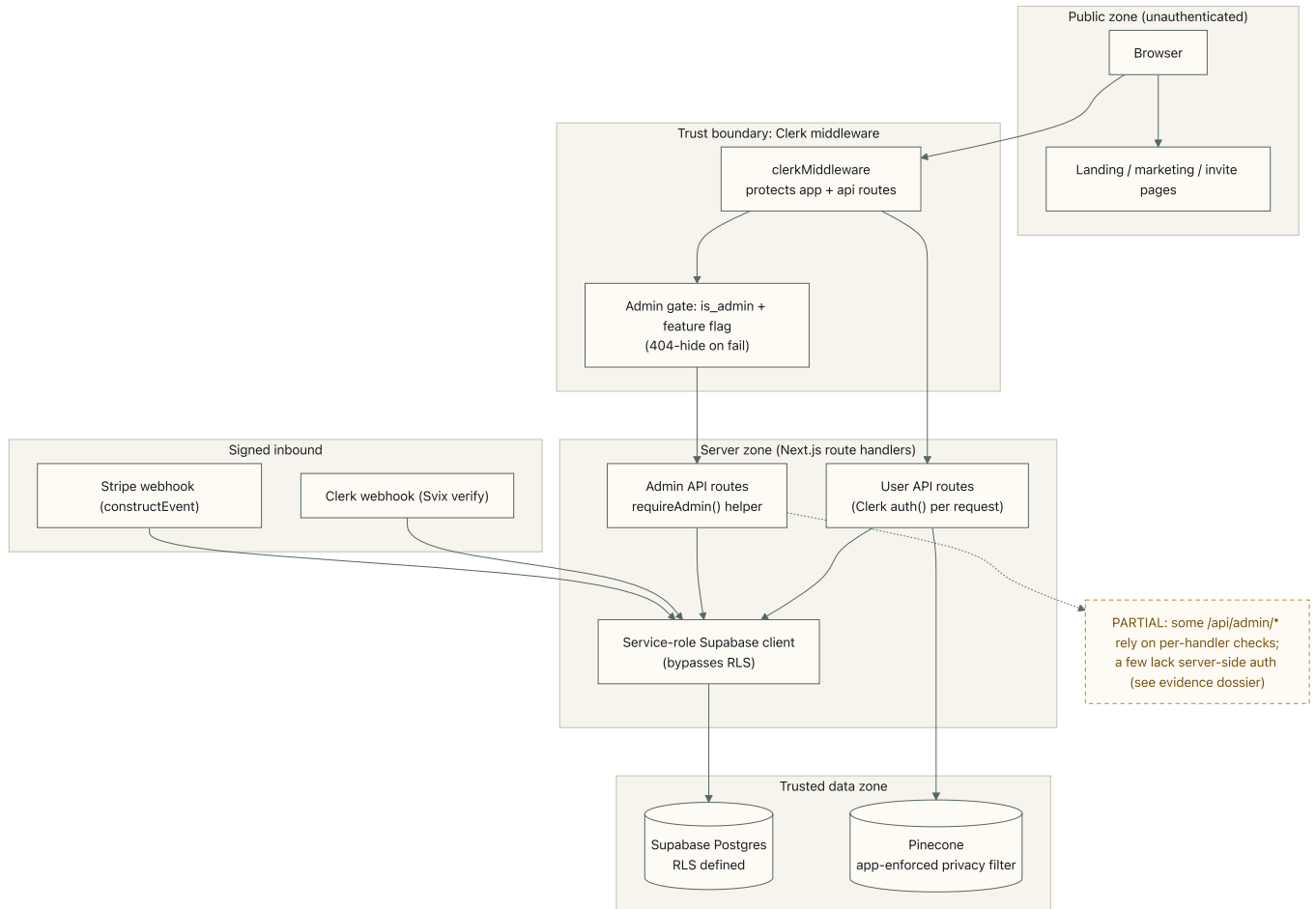


End-to-end data flow. Solid nodes exist today; dashed red nodes are recommended extensions.

06 Trust Boundaries

WHERE ENFORCEMENT HAPPENS

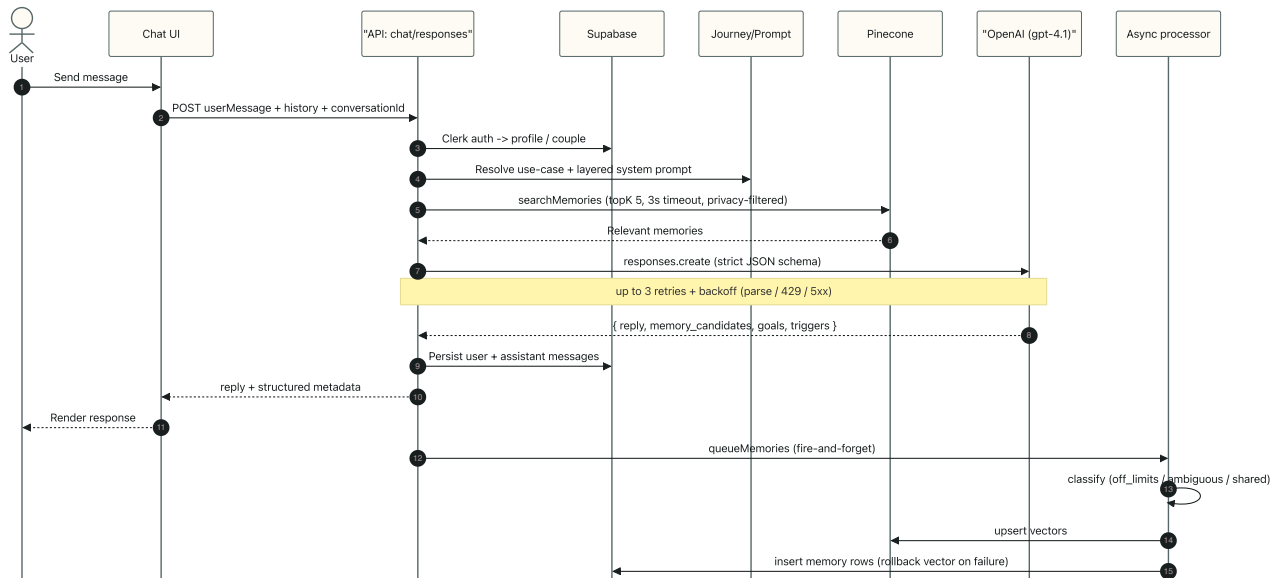
Clerk middleware is the primary trust boundary for the app and API surface; admin pages sit behind an additional `is_admin` + feature-flag gate that 404-hides on failure. Server handlers use a service-role Supabase client (bypassing RLS), so access control is enforced in the API layer. Signed inbound webhooks (Clerk via Svix, Stripe via `constructEvent`) are verified before touching data.



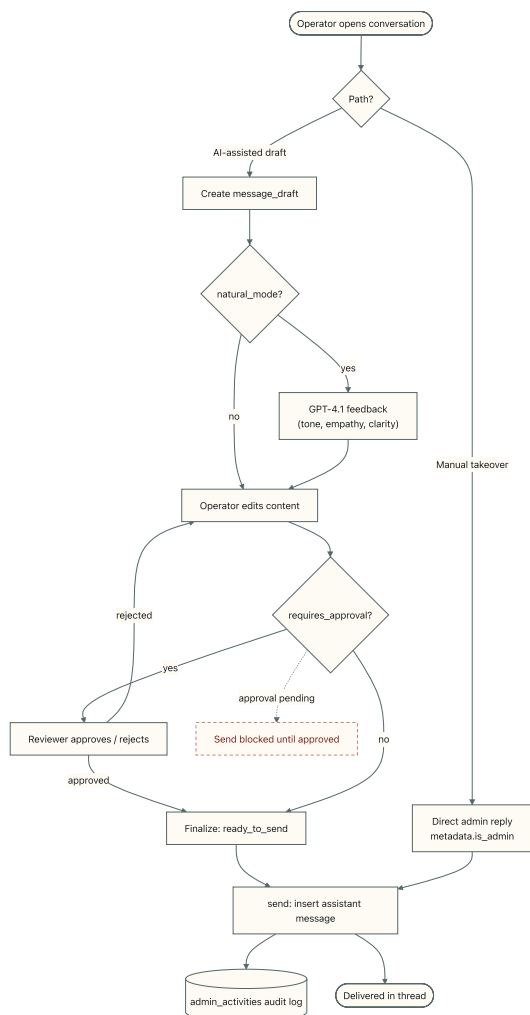
Trust boundaries. The amber note marks the partial admin-API authorization gap detailed in section 12.

07 Critical End-to-End Workflows

SEQUENCE & CONTROL FLOW



One chat turn: auth, prompt assembly, memory retrieval, structured LLM call, persist, background extraction.



Operator draft / approval / takeover with audit logging.

Why these are the critical paths

- The **AI turn** is the product's core loop and its main reliability surface (retries, timeout, schema).
- The **HITL flow** is the safety surface: the `requires_approval` gate blocks auto-send until a human approves; a separate path allows direct manual takeover.
- Both persist to an audit trail (`admin_activities`), making operator actions traceable.

Failure handling

A Pinecone timeout degrades gracefully (reply proceeds without memory); an LLM parse failure triggers retry; a memory write that fails after the vector upsert rolls the vector back.

08 Most Advanced Implementations

RANKED BY DIFFICULTY X VALUE

1

Privacy-aware dual-store memory IMPLEMENTED PARTIAL PATHS

Extract, classify (shared/private/off_limits/ambiguous), embed, and store to Pinecone + Postgres with vector-rollback on SQL failure; retrieval filters by ownership and store. Guardrails route infidelity/abuse/self-harm to off_limits.

memory-manager.ts · pinecone-client.ts · classifiers.ts

2

Human-in-the-loop operator console IMPLEMENTED

Draft to approve to send state machine with a hard send-gate, manual takeover, iteration history, and an audit log.

admin/conversations/[id]/drafts/[draftId]/route.ts

3

Stateful journey orchestration IMPLEMENTED

Session state machine (0a-4b) driving per-session prompts, reciprocal partner nudging, and lifecycle emails; up to 13 layered prompt sources.

user_journey_state · journey-service.ts · chat/responses/route.ts

4

Resilient structured-output turn IMPLEMENTED

OpenAI Responses API with strict JSON schema, 3x retry + backoff, server-side conversation chaining, and a 3s non-blocking retrieval timeout.

chat/responses/route.ts · memory/types/index.ts

5

Consent-gated shareable moments IMPLEMENTED

Message-escrow: one partner shares a moment only after explicit consent, with delivery and revocation.

shareable-moments/** · migration 20260124

6

Billing lifecycle + signed webhooks IMPLEMENTED NO DEDUP

Checkout, portal, and a signature-verified webhook handling subscription and invoice events; idempotency/dedup not visible.

stripe/webhook/route.ts

7

Invitation to couple linking IMPLEMENTED

Tokenized invite, validation, atomic couple creation, and Partner B contextual onboarding from shared memory.

invitations/** · couple/initialize · partner-b-onboarding.ts

09 AI System Design

INPUTS, ORCHESTRATION, SAFETY, CONTROL

Inputs & orchestration

- Layered system prompt from up to 13 sources (DB base prompt, identity/attribution, memory context, partner context, journey/session, consent, completion criteria, extraction instructions).
- Memory retrieval from Pinecone (topK 5, privacy-filtered, 3s timeout, non-blocking).
- History via last-10 turns or server-side chaining (`previous_response_id`).

Outputs & storage

- Strict-schema JSON: reply + memory candidates + goal suggestions + UI triggers.
- Messages persisted to Postgres; memories embedded and written to Pinecone + Postgres in the background.

Guardrails, validation, retries

Schema	Strict <code>json_schema</code> on every call
Retries	3x on parse / 429 / 5xx, backoff, 60s budget
Privacy	Regex classifier → off_limits/ambiguous; default-private
Dedup	Text + semantic (>0.85) before store
Consistency	Vector rollback if SQL insert fails
Human control	Ambiguous memories → user clarification; operator approval on outbound

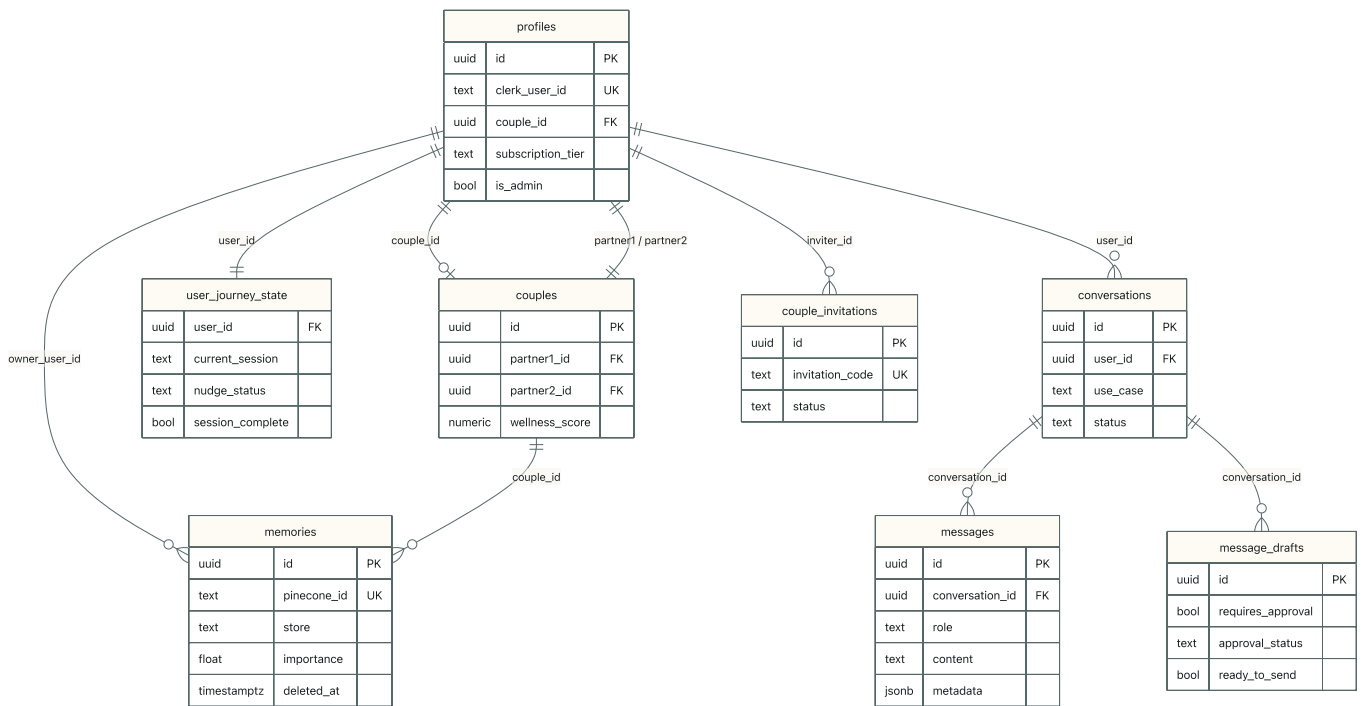
Recommended production architecture (Proposed)

Move background extraction to a **durable queue** (Upstash/QStash) so it survives cold starts; **activate the already-built hybrid retriever** (semantic + lexical + recency rerank in `hybrid-retriever.ts`, currently bypassed); add **response streaming**. Cost/observability: no token-usage tracking or eval harness is present today (**PROPOSED**).

Terminology kept precise: this is LLM orchestration with retrieval-augmented memory and structured outputs. There is no model training or fine-tuning, and no autonomous tool-calling loop.

10 Data Model & State

ENTITIES, TENANCY, TRANSITIONS



Core entities and key foreign keys (subset of ~37 tables).

Entity	Role & key state fields
profiles	Account, linked to Clerk (clerk_user_id); couple_id , subscription_tier , is_admin
couples	partner1_id / partner2_id , wellness_score
conversations / messages	Thread header + normalized turns (role , content , JSONB metadata)
memories	store , type , is_shared , importance , pinecone_id , soft-delete
message_drafts	requires_approval , approval_status , ready_to_send , sent_at
user_journey_state	current_session , nudge_status , session_complete

Tenancy & integrity: access is scoped by user_id / couple_id in API routes using the service-role client. RLS policies exist but are largely bypassed since identity is Clerk, not Supabase Auth (**INFERRED** design choice). Vector/metadata consistency is maintained by pinecone_id and rollback-on-failure.

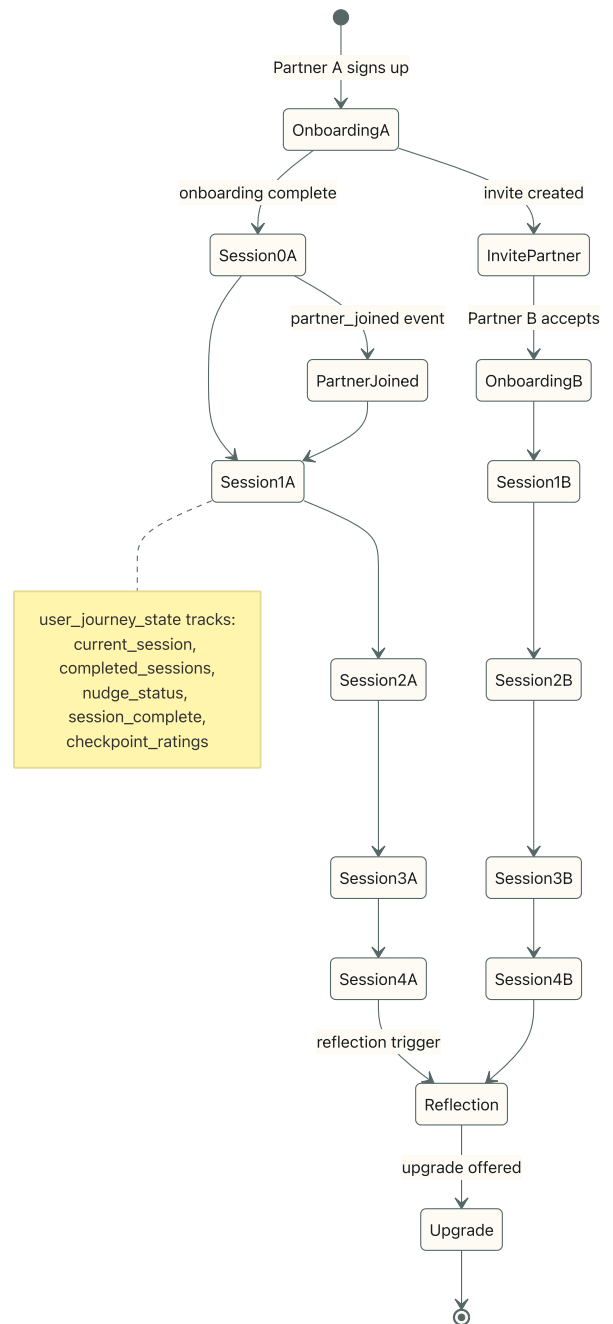
11 Journey State Machine

LIFECYCLE & STATUS ENUMERATIONS

State is explicit and queryable - the substrate any workflow engine needs. The journey machine drives which session prompt is used, when follow-ups fire, and when an upgrade is offered.

Representative status enumerations

Field	Values
journey.current_session	session_0a..4a / 1b..4b
journey.nudge_status	active / dormant / completed
drafts.approval_status	pending / approved / rejected
invitations.status	pending / accepted / cancelled / expired
goals.status	idea / active_working / achieved / dormant
scheduled_emails.status	pending / sent / failed / cancelled
memories.store	shared / private / off_limits / ambiguous



User-journey lifecycle.

12 Security, Privacy & Authorization

BOUNDARIES & GAPS

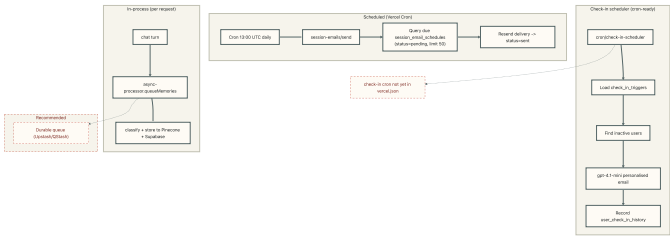
Control	Status	Detail
Authentication	IMPLEMENTED	Clerk middleware; protected route matchers
Admin page gating	IMPLEMENTED	<code>is_admin</code> + feature flag; 404-hide to conceal existence
Admin API authorization	PARTIAL	Per-handler <code>requireAdmin()</code> ; several <code>/api/admin/*</code> lack server-side auth
Webhook verification	IMPLEMENTED	Clerk via Svix; Stripe <code>constructEvent</code>
Memory privacy	IMPLEMENTED	Store classification + ownership filters; default-private
Secret handling	IMPLEMENTED	Service-role key server-only; anon key client-side
RLS enforcement	INFERRED	Policies defined but bypassed via service role; enforced in API layer
Input validation	PARTIAL	Manual type/enum/UUID checks; no schema library (Zod)
Rate limiting / CORS	PARTIAL	No rate limiting; CORS <code>*</code> on <code>/api/*</code>

Primary security risk to address first

Inconsistent server-side authorization on some admin API routes. The 404-hidden admin pages reduce discoverability, but API routes should each enforce `requireAdmin()`. This is called out plainly and is the top remediation item.

13 Production Readiness & Operations

DEPLOY, JOBS, MONITORING



Scheduled, cron-ready, and in-process background work.

Area	Status
Deployment (Vercel standalone, region pin)	IMPL
Image opt + long-cache headers	IMPL
Migrations (38, chronological)	IMPL
Health endpoint	PARTIAL
Session-email cron	IMPL
Check-in cron (send TODO, unregistered)	PARTIAL
Durable queue	PROP
Central logging / Sentry	PROP
Build type/lint gates	DEPRECATED

Operational maturity, honestly

The deploy spine (standalone build, region, caching, one real cron, deploy hooks) is solid. The gaps are the standard "harden for scale" backlog: durable jobs, observability, re-enabled build gates, and registering the second cron. The `vercel.json` function block also references stale routes (`api/ai/chat` , `api/memory/extract`) that no longer exist.

14 Testing & QA Evidence

WHAT EXISTS, WHAT DOES NOT

Check	Result	Detail
Type-check (<code>tsc --noEmit</code>)	39 ERRORS	20 in admin components, 8 in scripts; core AI/memory clean; non-blocking by config
Lint (<code>next lint</code>)	910 WARN, 1 ERR	Mostly no-explicit-any + unused imports; deferred at build
Production build	INCONCLUSIVE	Compile did not complete in this resource-contended sandbox (see BUILD_NOTES)
Automated tests	NONE	0 test files; no Jest/Vitest/Playwright config
Manual diagnostics	PRESENT	14 scripts (partner-flow, conversations, email-system, couple-store)

Highest-leverage QA gap (Proposed)

There is no automated test suite or CI. Given the privacy-critical memory classifier and the approval-gate logic, targeted unit tests around `classifiers.ts` and the draft `send` guard, plus a smoke E2E of the chat turn, would materially de-risk changes. Re-enabling the build's type/lint gates would prevent silent regressions.

15 Engineering Judgment & Intellectual Work

EVIDENCE-BASED

Capability	Repository evidence	Why it is demanding	Confidence
Abstraction & data modeling	Four-value memory store; journey enum; consent escrow	Encoding privacy + relationship semantics into a schema	High
Probabilistic-system reasoning	Strict schema + retries + timeout + rollback	Making an unreliable LLM dependency safe and consistent	High
Security/privacy reasoning	Default-private classifier; 404-hidden admin; service-role isolation	Designing for leakage prevention, not just features	High
Systems thinking	Coherent flow across auth, DB, vector, LLM, billing, email	Coordinating many surfaces into one product	High
Reliability/operational thinking	Graceful degradation; audit log; lifecycle cron	Planning for partial failure and operations	Moderate
Product/business reasoning	Operator console sized to over half the backend	Recognizing AI products need human oversight to ship	High
Debugging & failure analysis	Commit history: race-condition, redirect-loop, privacy fixes	Diagnosing distributed/async bugs over time	Moderate
Ownership under ambiguity	528 solo commits across the whole stack	Carrying an integrated product end to end	High

Confidence reflects how directly the repository supports each claim. No psychological or IQ-style assessment is made; only demonstrated engineering behaviour.

16 Professional Skill Matrix & Ownership

WHAT THIS PROVES, FOR WHICH ROLES

Target role	What this repository demonstrates
Senior full-stack engineer	End-to-end Next.js/React/TS product with 98 API routes, real data modeling, and third-party integration depth
AI application engineer	LLM orchestration with structured outputs, RAG-style memory, guardrails, retries, and human oversight
Product engineer	Domain-driven modeling, onboarding/journey flows, and an operator console that reflects how the product is actually run
Founding engineer	Sole ownership across frontend, backend, data, AI, billing, and deployment; shipped and hardened over 7.5 months
Solutions architect	Clear trust boundaries, integration topology, and pragmatic tradeoffs with a labelled hardening roadmap
Technical founder	Product/business reasoning (privacy as the wedge), monetization, and an operational control plane

Ownership & leadership evidence

- **Sole author** of 528 commits (git shortlog); no other contributors. Frontend, backend, schema, AI, billing, and Vercel deploy all authored by one person.
- **Inherited/generated/vendor code:** none identified as third-party-authored beyond standard npm dependencies and framework scaffolding. AI-assisted authorship, where used, is judged by the resulting architecture (evaluated throughout this document), not by volume.
- **Sustained ownership:** post-launch hardening commits (CVE patch, privacy and race-condition fixes) show responsibility beyond initial delivery.

17 Value, Limitations & Future Architecture

HONEST AND FORWARD-LOOKING

Commercial value & defensibility

- Privacy guarantee is the wedge: hard to copy credibly, builds trust.
- Memory compounds with use, aiding retention.
- Operator console enables safe scaling of AI output.
- Subscription billing is in place. **UNVERIFIED**: any revenue/user metrics.

Honest limitations & risks

- Some admin API routes lack server-side auth. **PARTIAL**
- In-process memory queue is non-durable. **PARTIAL**
- No automated tests / CI. **PROPOSED**
- Build type/lint gates disabled. **DEPRECATED**
- No rate limiting; CORS *. **PARTIAL**
- Stale `vercel.json` function paths. **DEPRECATED**

Future architecture (Proposed)

1) Durable queue for memory extraction; 2) activate the hybrid retriever in the chat path; 3) response streaming; 4) uniform admin-API authorization + rate limiting; 5) test suite + CI with re-enabled build gates; 6) token-usage/cost tracking and an eval harness for the privacy classifier. Each is incremental on the current design.

18 Resume-Ready Achievements & Interview Narratives

DEFENSIBLE FROM
EVIDENCE

Technical / product-engineering bullets

- Built a privacy-first AI companion on Next.js 15 / React 19 with 98 API routes and a 45-component operator console, integrating Clerk, Supabase, Pinecone, OpenAI, Stripe, and Resend.
- Designed a dual-store memory system (Pinecone + Postgres) with classifier guardrails and vector-rollback to prevent leaking one partner's private content.
- Shipped a human-in-the-loop console with a draft-approve-send state machine, manual takeover, and an audit trail.
- Hardened an LLM turn with strict structured outputs, 3x retry + backoff, and a non-blocking 3s retrieval timeout.

Founder / technical-leadership bullets

- Sole engineer across 528 commits and ~7.5 months, owning the full product stack and deployment.
- Set a privacy-first architecture (default-private memory, consent-gated sharing, 404-hidden admin) as the product's defensibility.
- Built the operational control plane needed to run an AI product safely with a human in the loop.

Interview narratives

- **Hardest architecture:** two-store memory consistency + privacy, solved with classification, ownership filters, and vector rollback.
- **Reliability:** made a slow/fallible LLM safe via schema, retries, and a non-blocking retrieval timeout.
- **Best product decision:** default-private classification + consent-gated sharing as the trust foundation.
- **End-to-end ownership:** invitation to couple-linking to Partner B contextual onboarding across API, DB, memory, prompts.
- **Lesson/tradeoff:** the in-process queue and unwired hybrid retriever are conscious scope tradeoffs and the clear next step.

19 One-Page Summary & Evidence Index

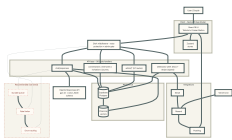
THE WHOLE STORY, BRIEFLY

What it is

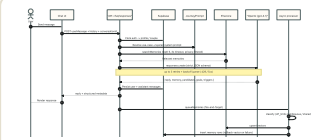
A privacy-first relationship-wellness AI companion for couples, with a human-operated control plane. Advanced pre-production SaaS.

What the architecture demonstrates

LLM orchestration with private memory/RAG, human-in-the-loop approval, stateful journeys, billing, signed webhooks, and operations tooling.



Architecture



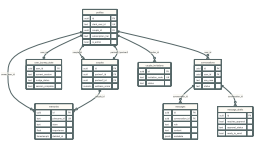
AI turn



Human-in-the-loop



Trust boundaries



Data model



Journey state



Background jobs



System context

Evidence index (selected)

- Auth + admin gate: `src/middleware.ts`
- Chat turn + retries: `api/chat/responses/route.ts`
- Memory + rollback: `lib/memory/memory-manager.ts`
- Privacy classifier: `lib/memory/classifiers.ts`
- HITL send-gate: `admin/.../drafts/[draftId]/route.ts`
- Journey: `migrations/20250117_...journey...sql`
- Shareable moments: `migrations/20260124_...sql`
- Stripe webhook: `api/stripe/webhook/route.ts`
- Cron: `vercel.json` + `session-emails/send`
- Counts: `report-data.json`

Full dossier: `portfolio-showcase/TECHNICAL_EVIDENCE.md`. Verification log: `BUILD_NOTES.md`.